

An Evolutionary Method for Active Learning of Mobile Robot Path Planning

Byoung-Tak Zhang

Dept of Computer Engineering
Seoul National University
Seoul 151-742, Korea
btzhang@comp.snu.ac.kr

Sung-Hoon Kim

Dept of Computer Engineering
Konkuk University
Seoul 143-701, Korea
kimsh@pluto.konkuk.ac.kr

Abstract

Several evolutionary algorithms have been proposed for robot path planning. Most existing methods for evolutionary path planning require a number of generations for finding a satisfactory trajectory and thus are not efficient enough for real-time applications. In this paper we present a new method for evolutionary path planning which can be used on-line in real-time. We use an evolutionary algorithm as a means for active learning of a route map for the path planner. Given a source-destination pair, the path planner searches the map for a best matching route. If an acceptable match is not found, the planner uses another evolutionary algorithm to generate on-line a path for the source-destination pair. The overall system is an incremental learning planner that gradually expands its own knowledge suitable for path planning in real-time. Simulations have been performed in the domain of robotic soccer to demonstrate the effectiveness of the presented method.

1 Introduction

The goal of robot path planning is to find a trajectory of a robot from a starting position to a goal position, while avoiding collision with obstacles. Recently, a number of path planning algorithms have been developed. The main difficulties in finding an optimal path arise from the fact that the analytical methods are too complex to be used in real-time and enumerative search methods are overwhelmed by the size of the search space. On the other hand, evolutionary algorithms have been shown to be effective in path planning when the search space is large.

Several evolutionary algorithms have been proposed for robot path planning. Khoogar and Parker [5] used genetic algorithms for obstacle avoidance of

a planar robot. Ram et al. [8] applied simple genetic algorithms to a mobile robot navigation problem in a 2-D space with stationary obstacles. Toogood et al. [11] studied path finding methods for a 3-DOF robot manipulator using a genetic algorithm. Most of these methods for evolutionary path planning require a number of generations for finding a satisfactory path and thus are not efficient enough to be used in real-time applications, such as robotic soccer. Kuc et al. [3] proposed a local path planning method to direct the motion at each instant of the trajectory in order to minimize the computation time for real-time obstacle avoidance.

In this paper we present a new method for evolutionary path planning which can be used on-line in real-time. The basic idea is to use an evolutionary algorithm as a means for active learning of a route map for the path planner. Given a source-destination pair, the path planner searches the map for a best matching route. If an acceptable match is not found, the planner uses another evolutionary algorithm to generate on-line a path for the source-destination pair. The overall system is an incremental learning planner that gradually expands its own knowledge suitable for path planning in real-time. Simulations have been performed in the domain of robotic soccer to demonstrate the usefulness of the presented method. Robotic soccer provides an excellent test-bench for studying various techniques of computational intelligence.

The paper is organized as follows. Section 2 briefly describes the robotic soccer simulator on which the experiments have been performed. An overview of the proposed evolutionary planning method is provided. Sections 3 and 4 describe the evolutionary algorithms in detail and show experimental results. Section 5 draws conclusions.

2 Overview of the Approach

2.1 The Simulation Environment for Robotic Soccer

Robotic soccer games are examples of problems in the real world which is moderately abstracted. The robotic soccer provides an excellent test-bench for studying various techniques of computational intelligence applied to mobile robotics. For the experiments of our evolutionary path planning method, we have implemented a simulator for robotic soccer according to the laws of the game for MIROSOT, The Micro-Robot World Cup Soccer Tournament [6].

The size of a robot is 7.5cm x 7.5cm x 7.5cm. The robots move around on a 2-D grid of 130cm x 90cm which is occupied by five robots or obstacles. The positions of the robots are determined by a vision system. Figure 1 shows the simulation environment used for the experiments.

We distinguish four different levels of planning. They are called strategies, tactics, routes, and motions. In robotic soccer, the strategic planner decides a mode of game, such as the offence or defence mode. The tactical planner generates short-term goals to realize the current strategy. Tactics are represented as pairs of the starting and destination positions for robot navigation. The third level of planning is route generation. This determines the via-points for the ball or robots to be moved to reach the destination from the starting position. The fourth level decides the motions to be taken by robots for route tracking.

The focus of this paper is on the two intermediate levels of tactical and route planning. The rest two components are assumed available for the performance of the whole system.

2.2 Route Generation

A route is a sequence of points representing a trajectory for the robot to track. The task of the route planner is to generate a route or n via-points given a pair (s, d) of the starting and destination positions. We denote a route as

$$\begin{aligned} R &= \langle r_0, r_1, r_2, \dots, r_{n+1} \rangle \\ &= \langle (r_{0x}, r_{0y}), (r_{1x}, r_{1y}), \dots, (r_{(n+1)x}, r_{(n+1)y}) \rangle \end{aligned}$$

where $r_i, i = 0, \dots, n+1$ are positions on the playground. A feasible route is a route R which satisfies the following conditions:

- r_0 and r_{n+1} are the same as the starting position s and the destination d , respectively, and
- the trajectory does not collide with obstacles, i.e. $r_i \notin O_j$ for $i = 1, \dots, n$ and $j = 1, \dots, N_{obj}$, where

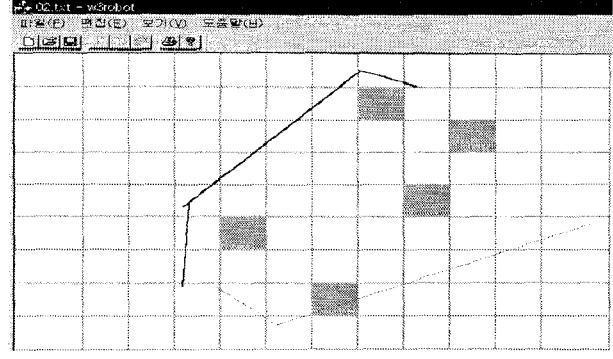


Figure 1: The simulation environment for robotic soccer. Also shown are examples of generated routes, one feasible and the other infeasible.

O_j is the set of points occupied by the j th obstacle.

A route is said to be optimal if it is the shortest feasible path from s to d . Figure 1 shows two examples of routes, one feasible and the other infeasible.

2.3 Tactics Learning

The objective of the tactical planner is to collect a set of effective tactics which implements a strategy produced by the strategic planner. In the offensive strategy mode, for example, an effective set of tactics should contain source-destination pairs near the goal posts of the adversary team.

Tactics are simply represented as (s^i, d^i) pairs, each of which designates the starting and destination positions of robot navigation. Tactics can be realized by a number of routes. Figure 2 shows two feasible routes for a single tactics.

A meaningful set of tactics constitute a strategy. A set of tactics is called a tactics base TB if the tactics are distinctive and each is realized as an optimal route, i.e.

$$TB = \{T_1, T_2, T_3, \dots, T_{N_{tactics}}\} \quad (2)$$

where $T_i = (s^i, d^i)$ and for each $j = 1, \dots, N_{tactics}$ ($i \neq j$): $s^i \neq s^j$, $d^i \neq d^j$ or both.

We define a route base RB to be the set of optimal routes, each of which corresponds to an element in the tactics base:

$$RB = \{R_1, R_2, R_3, \dots, R_{N_{route}}\} \quad (3)$$

where $R_i = \langle s^i, r_1^i, r_2^i, \dots, d^i \rangle$ is an optimal route for the tactics $(s^i, d^i) \in TB$.

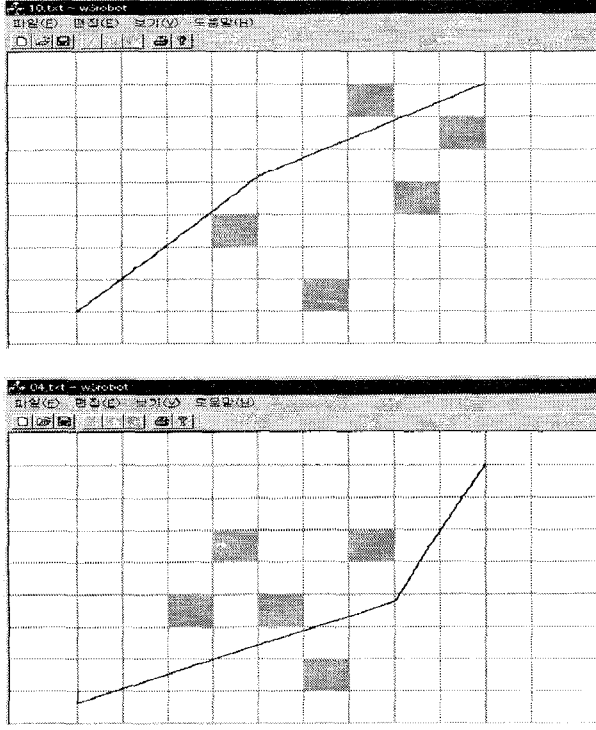


Figure 2: Two feasible routes for a single tactics (s, d) of a starting position and a destination.

This paper explores an evolutionary method for automatically building a tactical planner which can be used on-line in real-time. We first present an evolutionary algorithm for solving the routing problem. We then present another evolutionary algorithm for building a tactical planner.

3 Route Generation Using an Evolutionary Algorithm

The positions on the field are represented as a pair of x - y coordinate values normalized into the interval $[0, 1]$. This encoding scheme of using real-valued genes is contrasted with most existing evolutionary methods for robot path planning in which binary strings are used for chromosomes.

Given a pair, (s, d), of a starting and destination positions, the algorithm first generates a population RC of M_{route} candidate routes from s to d with random via-points:

$$RC = \{R_1, R_2, \dots, R_{M_{route}}\} \quad (4)$$

where $R_i = \langle s^i, r_1^i, r_2^i, \dots, r_n^i, d^i \rangle$ is the i -th route with n via-points $r_1^i, r_2^i, \dots, r_n^i$.

Each route is evaluated on its fitness. The fitness of the i th route R_i is defined as the sum of the lengths of segments on the route:

$$F(R_i) = \sum_{j=0}^n \|r_j^i - r_{j+1}^i\| \quad (5)$$

with $r_0^i = s^i$ and $r_{n+1}^i = d^i$.

The trajectories are ranked according to their fitness value. The best τ percent of t th population $RC(t)$ are then selected for crossover and mutation to generate the next generation $RC(t+1)$ of M_{route} routes. The best route of generation t is substituted for a randomly chosen individual in $RC(t+1)$.

Crossover between two parent routes is performed by splitting each at a segment boundary chosen at random. Crossover and mutation may generate infeasible routes. In this case, the values are repaired to lie near the obstacle and satisfy the feasibility. Mutation takes a route and changes each coordinate value with a probability of μ_{mut} . We use the exponential mutation scheme [7]. The new value is generated by adding a random value:

$$\Delta = \pm I \cdot 2^{-\epsilon \cdot S} \quad (6)$$

where the sign of I is generated at random. The random number ϵ is chosen from a uniform distribution over $[0, 1]$. The mutated values are corrected to lie within the interval $[0, 1]$ if they are out of the interval.

The density function $\phi(Z)$ and the distribution function $\Phi(Z)$ for the random variable $Z = 2^{-\epsilon \cdot S}$ are given as:

$$\phi(z) = \frac{1}{S \cdot \ln(2) \cdot z} \quad (7)$$

$$\begin{aligned} \Phi(z) &= Pr(Z \leq z) = \int_{2^{-S}}^z \phi(u) du \\ &= 1 + \frac{1}{S \cdot \ln(2)} \cdot \ln(z) \end{aligned} \quad (8)$$

The exponential characteristic of $\delta = 2^{-\epsilon \cdot S} \in [2^{-S}, 1]$ ensures that the mutation step size of maximum value I is possible, but small steps are more frequent. The constant S determines the shape of the exponential function and thus influence the probability of choosing large mutation steps: the larger the value S , the less the probability of taking large steps. S also determines the smallest step size $I \cdot 2^{-S}$. This is contrasted with the evolution strategy and evolutionary programming in which mutation is performed by normal distribution with which very small steps or very large steps take place with small probability. In the experiments, we used $I = 0.2$ and $S = 3$.

Parameter	Value
population size	200
max generation	1000
crossover rate	0.4
mutation rate	0.2
truncation threshold	0.5
number of via-points	3
number of obstacles	5

Table 1: Parameters used for the experiments.

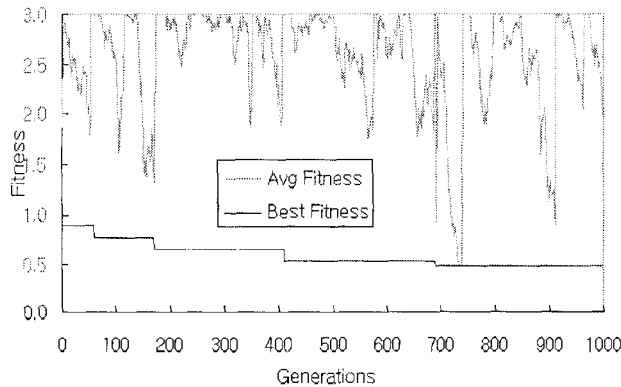


Figure 3: The evolution of fitness values as generation goes on.

The parameter values used in the experiments are summarized in Table 1. Figure 3 shows the best and average fitness values as generation goes on. The corresponding best routes are depicted in Figure 4.

4 An Evolutionary Algorithm for Active Learning of Tactics

In the last section, we have described an evolutionary algorithm and experimentally demonstrated that it can generate near-optimal routes for implementing given tactic. This method can be used on-line for path planning: whenever an instance of tactics is given, the evolutionary algorithm is executed to optimize the routes. However, this optimization can be a time-consuming process. The experiments indicate that it takes a few hundred generations, which takes at least a dozen minutes in wall-clock time. Many robotics tasks, including the robotic soccer game, require the planning be completed real-time or the movement delayed minimal.

One method to improve the speed of optimization is to prepare a set of tactics off-line in advance. Once a tactics base (with the associated route base) is con-

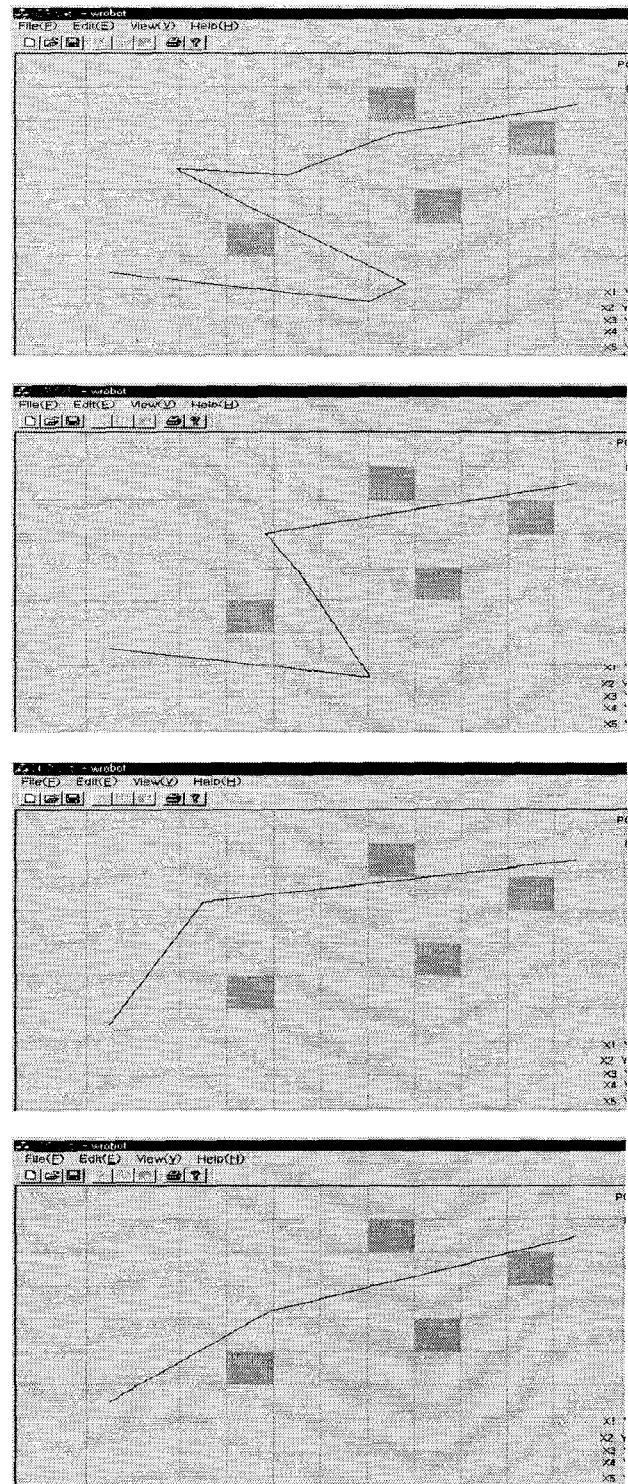


Figure 4: The best-of-generation routes found by the evolutionary algorithm. The plots are at generations $g = 1, 18, 26, 197$ from top to bottom.

structed, it can be used to find the route for an instance (s, d) of tactics by pattern matching, i.e. the best match is

$$T_{i*} = \min_{T_i \in TB} \{ D(T_c, T_i) \} \quad (9)$$

where $T_c = (s, d)$, T_i are elements of TB , and $D(T_c, T_i)$ is a distance measure. Then the route $R_{i*} \in RB$ that corresponds to the best matching T_{i*} is the best route for the tactics (s, d) .

We use a simulator to generate and solve problems to collect a set of useful tactics. This can be done by active learning [13, 2]. We focus on the learning task of ball-passing between arbitrary two points for a given configuration of obstacles. An evolutionary algorithm is used to explore the tactics space by generating novel tactics and selecting useful ones.

The algorithm starts with a tactics base of ν tactics initialized at random or by some other methods:

$$TB = \{ T_1, T_2, \dots, T_\nu \}. \quad (10)$$

Then a population of $M_{tactics}$ candidate tactics are created using genetic operators:

$$TC = \{ T_1, T_2, \dots, T_{M_{tactics}} \} \quad (11)$$

The candidate tactics with a large fitness value (defined below) are selected to expand the tactics base TB . Notice that we distinguish the population TC of candidate tactics from the tactics base TB .

To produce TC from TB , the fitter tactics are selected from TB to produce new tactics as candidates for the new generation of tactics base of increased size. New tactics are created by applying crossover and mutation operators to the selected parent tactics.

To promote the novelty of tactics we calculate the similarity of the candidate tactics T_c in TC with all of the existing tactics T_i in the current TB . The similarity of two tactics, T_c and T_i , is defined by the minimum distance between their corresponding routes, R_c and R_i , where R_i is the route corresponding to the tactics $T_i \in TB$ and R_c is the best route for the candidate tactics T_c found by the evolutionary algorithm described in the last section.

In effect, the fitness of T_c is defined as the minimum distance from R_c to $R_i \in RB$:

$$\begin{aligned} F(T_c) &= D(R_c, R_{i*}) \\ &= \min_{R_i \in RB} \{ D(R_c, R_i) \} \end{aligned} \quad (12)$$

where R_k is the best route for T_k . It should be noticed that measuring the fitness of candidate tactics requires

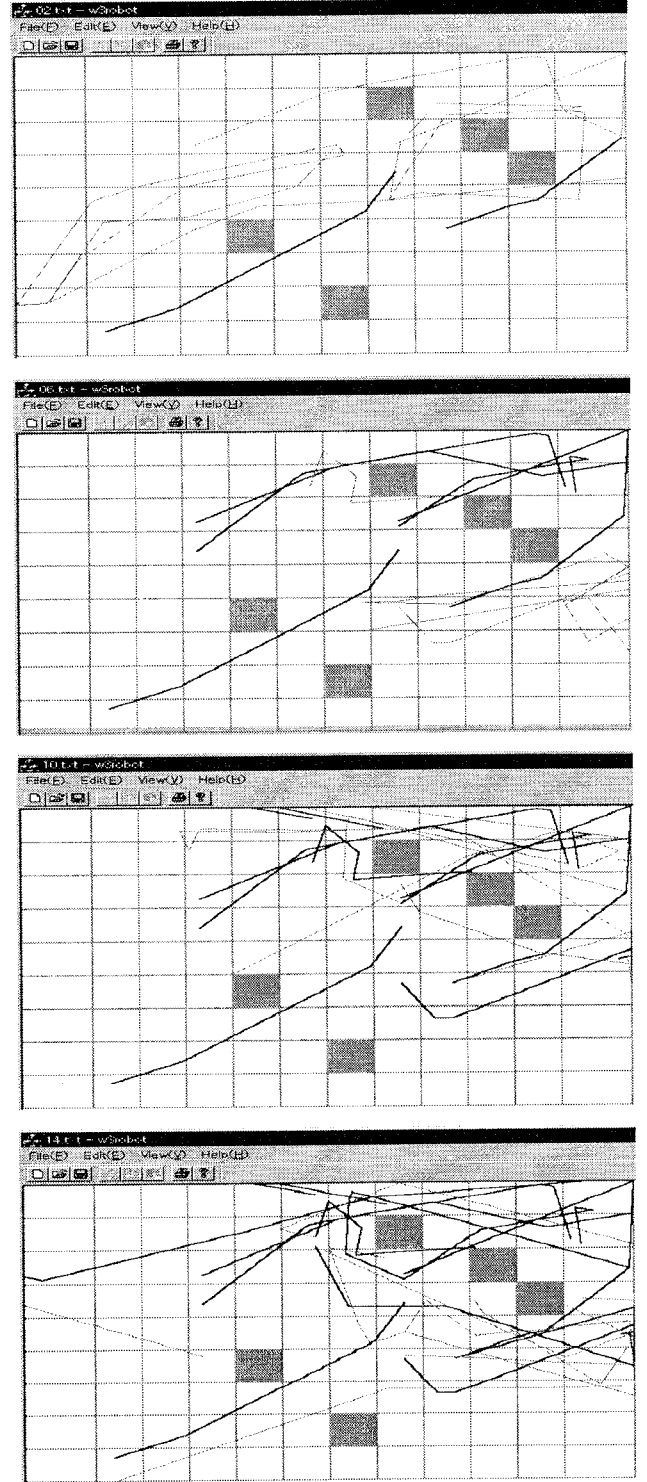


Figure 5: Active learning of tactics as generation goes on. Pictures, from top to bottom, show 2, 6, 10, and 14 tactics with their corresponding routes. The dotted lines show 10 candidate routes from which two are selected to expand the route base.

execution of the route generator, i.e. an evolutionary algorithm, described in Section 3 as a subroutine. This causes, however, no problem because it is an off-line process taking place when computing is almost free.

Figure 5 shows the evolution of the tactics base with its associated routes. Here the initial tactics base started with two tactics which was incremented each generation by two new tactics selected from 10 candidates. A tendency can be observed that offspring routes that are similar but have distinctive features are generated and integrated into the existing tactics base.

5 Conclusions

We have presented an active learning method for robot path planning using evolutionary algorithms. The performance of the method was demonstrated on a simulated robotic soccer. Experimental results support that the method is useful in automating the map building process for path planning and can be used to accelerate the optimization speed of existing evolutionary path planning techniques.

Our method is distinguished from other evolutionary approaches for robot path planning. One difference is in the encoding scheme. While most conventional methods use bitstrings, we encode the parameters directly with real-values, which is more natural for this problem and requires no extra decoding for fitness evaluation.

Another, more important, contribution of our approach is the introduction of the active learning concept to the robot path planning domain and its implementation using an evolutionary algorithm. The method seems promising to improve the real-timeness of evolutionary path planning and as a means for automatic skill acquisition in robotics problems.

Acknowledgments

This research was partially supported by grants from the Korea Science and Engineering Foundation (#96-0102-13-01-3) and the Korean Ministry of Information and Communications through ETRI.

References

- [1] Chien, Y.P., Koivo, A.J., and Lee, B.H., "On-line generation of collision free trajectories for multiple robots," in *Proc. of IEEE Int. Conf. on Robotics and Automation*, pp. 209-214, 1988.
- [2] Cohn, D. A., Ghahramani, Z., and Jordan, M. I., "Active learning with statistical models," in *Advances in Neural Information Processing 7*, Cambridge, MA: MIT Press, pp. 705-712, 1995.
- [3] Han, W.G., Baek, S.M., Kuc, T.Y., "Path planning of visual-servoed multiple mobile robots using genetic algorithms," in *Proc. 1996 Micro-Robot World Cup Soccer Tournament*, pp. 57-63, 1996.
- [4] Khoogar, A. R. and Parker, J. K., "Obstacle avoidance of redundant manipulators using genetic algorithms," in *Proc. IEEE Int. Conf. on Robotics and Automation*, 1991.
- [5] The MOROSOT Laws of the Game, *Final Program of the 1996 Micro-Robot World Cup Soccer Tournament*, 1996.
- [6] Mühlenbein, H. and Schlierkamp-Voosen, D., "The science of breeding and its application to the breeder genetic algorithm," *Evolutionary Computation*, 1(4):335-360, 1994.
- [7] Ram, A., Arkin, R., Boone, G. and Pearce, M., "Using genetic algorithms to learn reactive control parameters for autonomous robotic navigation," *Adaptive Behavior*, 2(3): 277-304, 1994.
- [8] Scheuer, A. and Fraichard, Th., "Planning continuous-curvature paths for car-like robots," in *Proc. IROS-96*, New York, NY: IEEE Press, pp. 1304-1311, 1996.
- [9] Stone, P. and Veloso, M., "Beating a defender in robotic soccer: memory-based learning of a continuous function," in *Advances in Neural Information Processing 8*, Cambridge, MA: MIT Press, pp. 896-902, 1996.
- [10] Toogood, H., Hao, H., and Wong, C., "Robot path planning using genetic algorithms," in *Int. Conf. on Syst. , Man, and Cybernetics*, pp. 489-494, 1995.
- [11] Zhang, B. T., "Learning by incremental selection of critical examples," *GMD Technical Report No. 735*, German National Research Center for Information Technology, St. Augustin/Bonn, Germany, 1993.
- [12] Zhang, B. T., "Accelerated learning by active example selection," *International Journal of Neural Systems*, 5(1):67-75, 1994.
- [13] Zhang, B. T. and Mühlenbein, H., "Balancing accuracy and parsimony in genetic programming," *Evolutionary Computation*, 3(1):17-38, 1995.